

Ejecución local, sandbox y por qué los hooks ahorran tokens

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 6 de 12 · Notas en español (es-ES)

El argumento de tokens de la sesión 6 es cuantitativo, no estético. Pedirle a Claude que vuelva a emitir un fichero ya escrito para que comillas y saltos de línea coincidan con la guía de estilo es un segundo pase de generación. Ejecutar Prettier (o cualquier formateador/linter) en el **sandbox** después de Write es CPU local. El orden de magnitud del formador para una generación completa: del orden de **500–2000 tokens ahorrados** al no hacer que el modelo reprocese el formato. Los hooks son el mecanismo; el sandbox es donde esos comandos corren de verdad.

EL SANDBOX LO GESTIONA EL PRODUCTO

No traéis un sandbox propio. El formador: Claude lo gestiona; no hay sandbox a medida. Las instalaciones de plugin a nivel de usuario caen en el espacio de usuario; las de proyecto, en sandboxes de proyecto. Las rutas difieren en Mac y Windows y no se resolvieron del todo en directo: fiaros de **Claude Code** vigente, no de un directorio home recordado.

Qué significa «local» aquí

Echo, NPX y los scripts de shell se ejecutan en la máquina/espacio de trabajo donde corre Claude Code, dentro del mismo entorno aislado que ya guardaba temporales MCP y árboles generados (la analogía tipo Docker de la sesión 4). Ese aislamiento es por lo que los formateadores pueden tocar la copia de trabajo sin reenviar el fichero entero al modelo. También es por lo que un hook que funciona en la sesión de terminal puede no verse en el webview del IDE hasta recargar la ventana / recargar webviews (estilo Ctrl+Shift+P) o activar el ajuste preview de hooks.

Dónde se siguen gastando tokens

- CLAUDE.md y las skills siguen añadiéndose a las peticiones al modelo: es contexto de pago, aunque hayáis sacado Prettier.
- Los bucles de verify/código limpio (módulos 04–05) se facturan; desactivadlos en memoria si vais a testear en local.
- Adjuntar toda la carpeta backend en vez de `openapi.yaml` se factura.
- Diagramas JPEG/SVG en vez de Mermaid se facturan (sesión 4).
- Los resultados de tools MCP ocupan contexto mientras están en la ventana, aunque la obtención sea una llamada a tool.
- Un hook que *imprima una novela en el prompt* en cada tool use sería un mal hook: dejad la salida corta.

Scripts como cuerpo de la política local

Los hooks no se limitan a `npx prettier`. Cabe todo lo que un script de shell pueda hacer: Python, otros binarios, `protect-files.sh`, `block-dangerous-commands.sh`. El formador guardó scripts bajo una carpeta `hooks/` del árbol de trabajo y los referenció desde `settings.json`. Los compañeros pidieron las cabeceras de comentarios de esos scripts para copiar la intención sin ingeniería inversa: esos ficheros se subieron al repo público de demo. Los comentarios del script son para personas y para vosotros; no necesitan vivir en el prompt del LLM.

Comando local	Evento de hook típico	Efecto en tokens
<code>npx prettier / eslint --fix</code>	post-tool-use en Edit/Write	Evita un pase de regeneración de formato (cientos-miles de tokens en una escritura grande).
<code>protect-files.sh</code> (denegar .env, package-lock.json, .git, secretos Prisma, ...)	pre-tool-use en Edit/Write	Evita reescrituras de ficheros secretos; también evita generaciones de «deshaz esto».
<code>block-dangerous-commands.sh</code> (rm, git imprudente, ...)	pre-tool-use en Bash	Corta desastres en modo auto; más barato que reparar un árbol borrado con el modelo.
echo de estado	post-tool-use	Negligible; solo observabilidad.

Los plugins siguen desempaquetándose en el sandbox

Cuando instaláis un plugin de marketplace o propio a nivel de proyecto, `settings.json` gana una referencia; Claude descarga el paquete y deja skills/agentes/hooks disponibles desde el sandbox. Puede que no veáis carpetas nuevas en el árbol git. La instalación a nivel de usuario va al espacio de usuario. Tratad los plugins como una biblioteca que consumís, no como ficheros que editáis a la ligera salvo que seáis colaboradores del repo origen (GitHub).

Porcentajes de examen que el formador ató a esta fase del curso

Aún el 13 de agosto, tras los hooks, mapeó pesos de Foundation (confirmad en Skilljar antes de presentaros): arquitectura y operaciones de agentes ~27 %, diseño de tools y MCP ~18 %, configuración y flujos de Claude Code ~20 %, ingeniería de prompts y salida estructurada ~20 %, gestión de contexto y fiabilidad ~15 %. Hooks, sandbox y tools locales conscientes de tokens caen en configuración/flujos y contexto/fiabilidad. No citéis los porcentajes como eternos; citad la idea de que el trabajo local determinista debe salir del bucle de pago.

Comprobación de estudio

Si un escenario pregunta cómo recortar coste en un repo Claude Code greenfield que ya tiene una guía de estilo estricta, la respuesta que puntúa: dejad las reglas de arquitectura en `CLAUDE.md`; moved formateador y scripts de guarda de ficheros a hooks en el sandbox; no enviéis la spec del formateador por el modelo en cada Write.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Sandbox, hooks y ámbitos de instalación de plugins.

- **Límites de tasa**

<https://docs.claude.com/en/api/rate-limits>

Los medidores de uso hacen operativa la historia de 500–2000 tokens, no académica.

- **Caché de prompts**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Función de API posterior; no sustituye a los hooks locales en Claude Code.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

El uso de la org y el acceso a modelos siguen acotando los tokens de pago restantes.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.